Delco

Banco de dados

Diagrama de relacionamentos e arquitetura de dados

Copyright (©) 2026 Erik Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 4

- MySQL é a fonte da verdade 4
- Áreas do modelo 4
- Banco suportado 4

Diagrama de relacionamentos 6

- Acesso 7
- Definições 7
- Execução 8
- Cardinalidades 8

Regras e convenções 9

- Convenções do modelo 9
- Integridade e concorrência 9
- Tipos usados 9

Tabelas 11

- users 11
- api_tokens 11
- audit_log 12
- workflows 13
- workflow_versions 13
- workflow_runs 14
- task_instances 15
- task_attempts 16
- task_logs 17
- workers 17
- django_migrations 18

Máquinas de estado 19

- Tarefa 19
- Execução 20

Uma execução passo a passo 21

Consultas úteis 21

Como parear o banco 22

Com Docker (recomendado) 22

MySQL próprio 22

Banco gerenciado 22

Redis 22

Backup e restauração 23

Retenção e tamanho 23

Problemas comuns 23

Evolução do esquema 24

Capítulo 1

Visão geral

Este documento descreve a arquitetura de dados do Delco: um único banco MySQL com 10 tabelas de negócio, definidas como modelos do Django em `delco/models.py`, e a tabela de controle de migrações do Django. Mostra o diagrama de relacionamentos, todos os campos com tipos e chaves, as regras de cada tabela, a máquina de estados das execuções e tarefas e como preparar o banco para usar o projeto.

MySQL é a fonte da verdade

Tudo o que importa vive no MySQL: definições, versões, execuções, tarefas, tentativas, logs, workers, usuários e auditoria. O Redis é só transporte (filas e eventos em tempo real): se ele for apagado, nenhuma execução se perde, porque o agendador compara o banco com as filas e republica o que faltar.

Onde	O que guarda	Pode perder?
MySQL	Estado completo e histórico	Não: faça backup
Redis Streams `delco:queue:<fila>`	Mensagens com o id da tarefa a executar	Sim: são republicadas
Redis pub/sub `delco:events`	Avisos de mudança para o painel	Sim: o painel relê a API
Redis `delco:wake` e `delco:load-errors`	Sinal para acordar o agendador e erros dos arquivos	Sim

Áreas do modelo

Área	Tabelas	Quem escreve
Acesso	`users`, `api_tokens`, `audit_log`	API
Definições	`workflows`, `workflow_versions`	Agendador (ao ler a pasta de workflows)
Execução	`workflow_runs`, `task_instances`, `task_attempts`, `task_logs`	API, agendador e workers
Infraestrutura	`workers`, `django_migrations`	Workers e o `migrate` do Django

Banco suportado

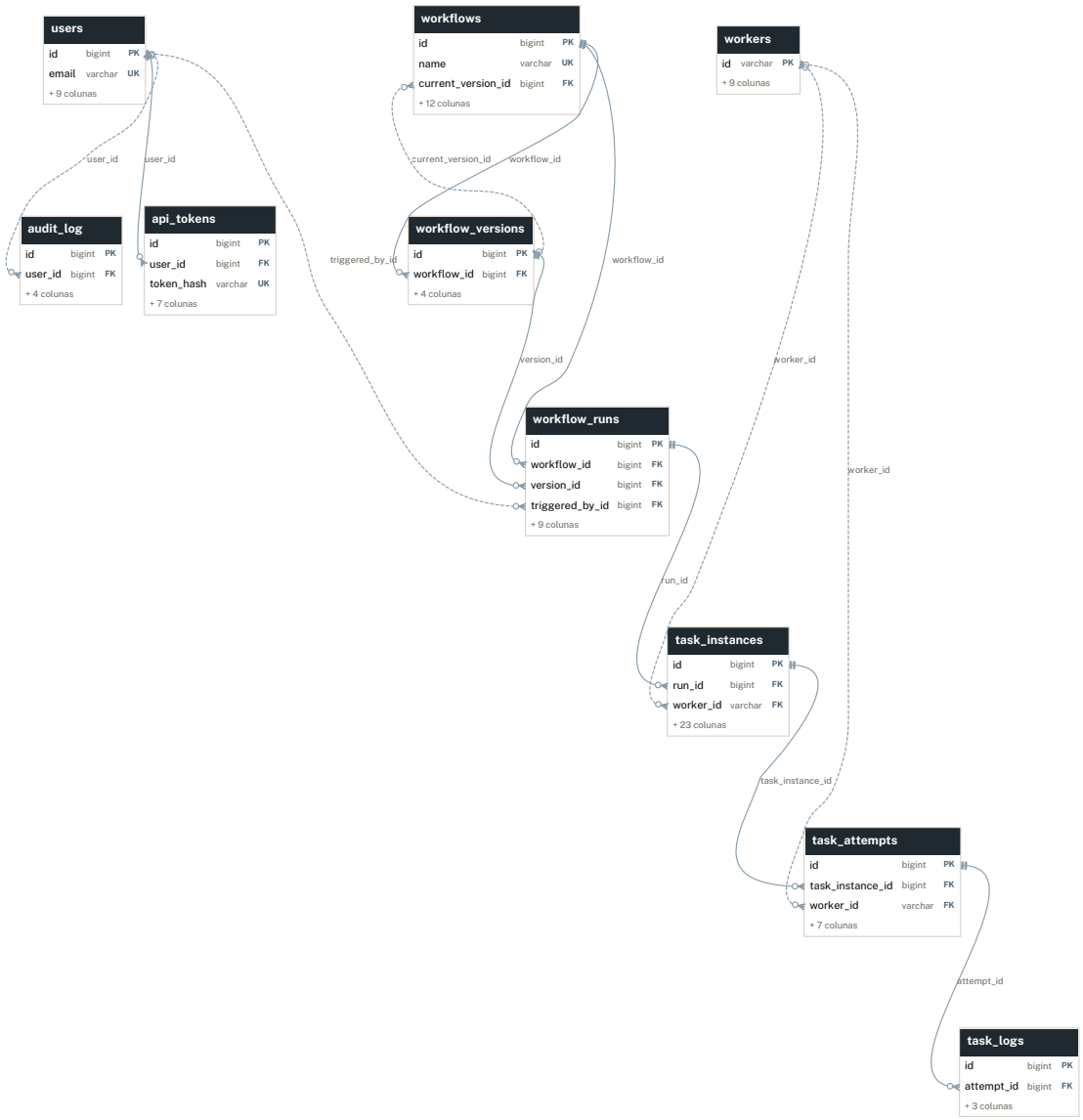
Item	Valor
MySQL	8.4 LTS (imagem `mysql:8.4`), o mínimo do Django 6.1; tabelas InnoDB
Codificação	utf8mb4 com a collation utf8mb4_0900_ai_ci
Fuso	Todas as datas em `datetime(6)` em UTC: a conexão roda `SET time_zone = '+00:00'` e o painel mostra no fuso do navegador

Item	Valor
Acesso	ORM do Django 6.1 sobre o mysqlclient 2.3, uma conexão por processo/vaga, isolamento READ COMMITTED; autocommit, com `transaction.atomic()` onde a escrita precisa ser atômica
Criação das tabelas	Migrações do Django: `delco migrate` ou `python manage.py migrate` (a API, o agendador e os workers também migram ao subir, com trava nomeada `GET_LOCK`)
Testes	Banco real separado (`delco_test`), recriado a cada sessão de testes

Capítulo 2

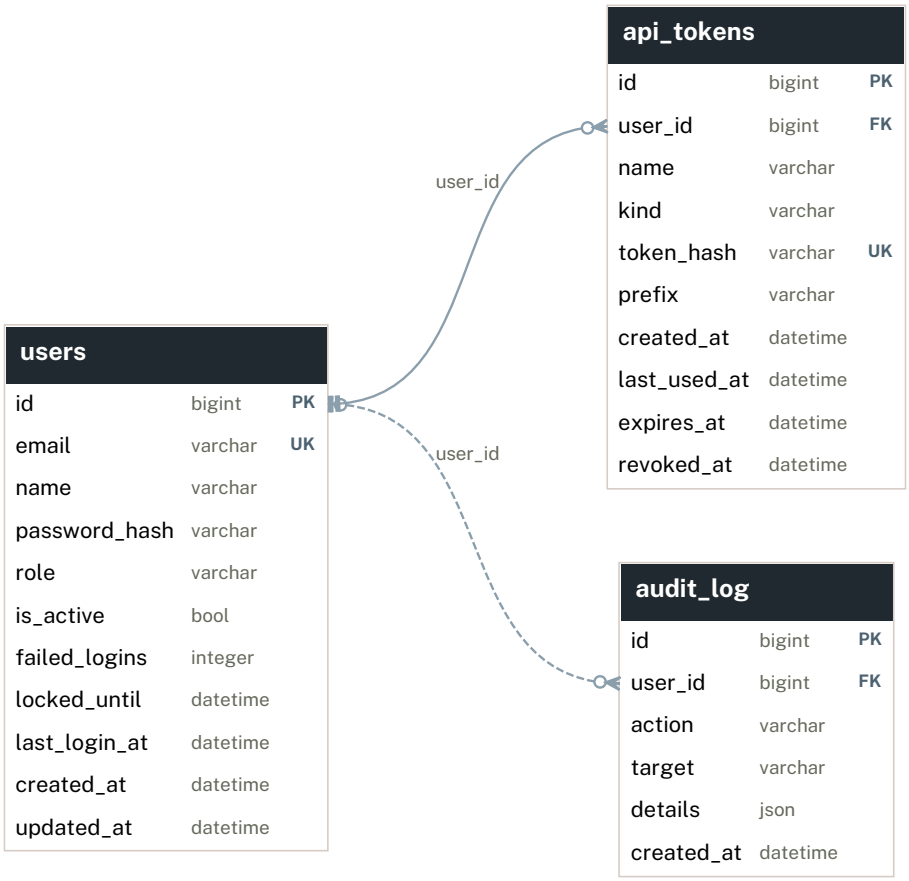
Diagrama de relacionamentos

As ligações são chaves estrangeiras de verdade. Linha cheia: relação obrigatória. Linha tracejada: opcional (a coluna pode ser nula, e vira nula se o registro pai for apagado). Na ponta, o pé de galinha indica o lado "muitos".



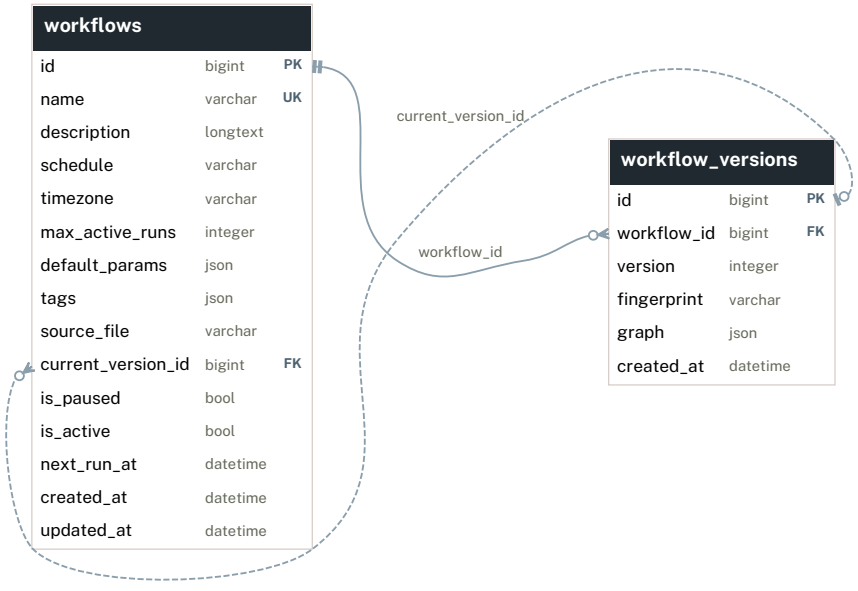
Todas as tabelas com chaves primárias, únicas e estrangeiras

Acesso



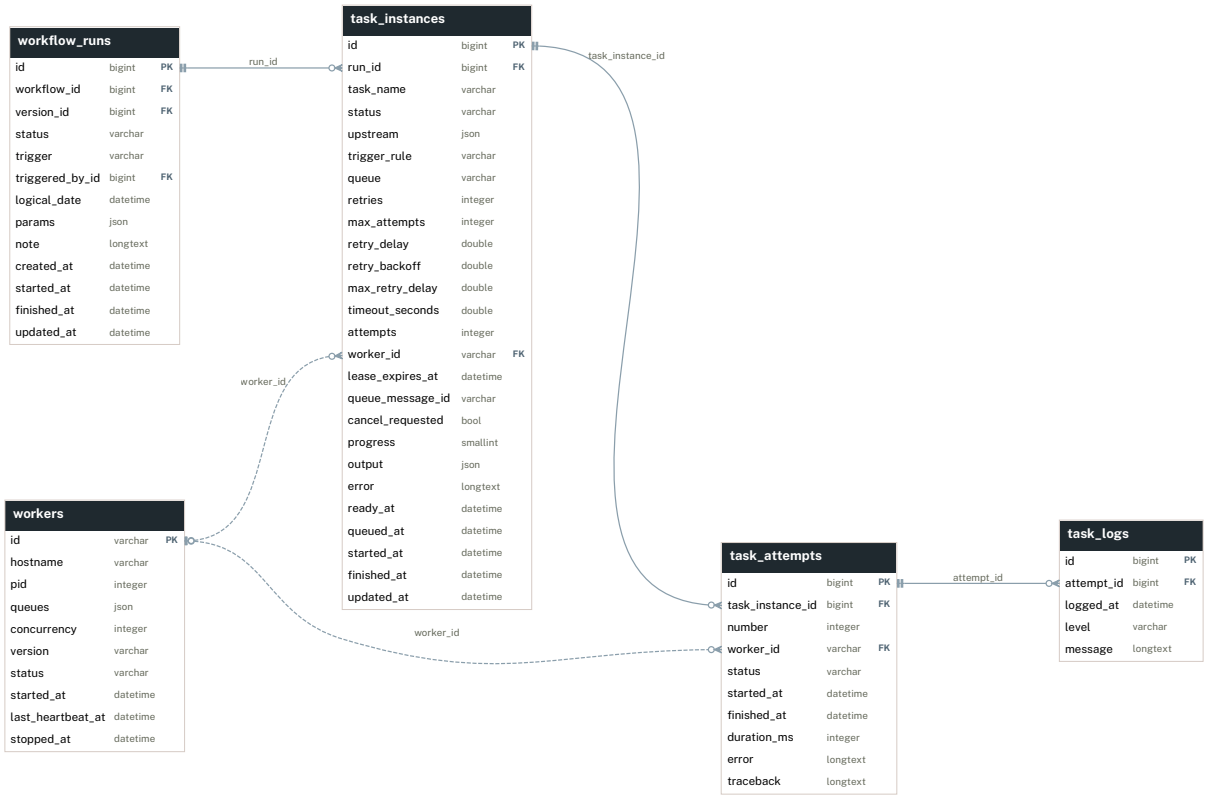
users 1:N api_tokens · users 1:N audit_log

Definições



workflows 1:N workflow_versions · workflows N:1 versão atual

Execução



workflow_runs 1:N task_instances 1:N task_attempts 1:N task_logs · workers 1:N tentativas

Cardinalidades

Pai	Filho	Coluna	Ao apagar o pai
users	api_tokens	user_id	apaga os tokens (CASCADE)
users	audit_log	user_id	mantém o registro, user_id vira nulo
users	workflow_runs	triggered_by_id	mantém a execução, triggered_by_id vira nulo
workflows	workflow_versions	workflow_id	CASCADE
workflow_versions	workflows	current_version_id	vira nulo
workflows	workflow_runs	workflow_id	CASCADE
workflow_versions	workflow_runs	version_id	bloqueia (só some junto com o workflow)
workflow_runs	task_instances	run_id	CASCADE
task_instances	task_attempts	task_instance_id	CASCADE
task_attempts	task_logs	attempt_id	CASCADE
workers	task_instances / task_attempts	worker_id	vira nulo

Capítulo 3

Regras e convenções

Convenções do modelo

- Nomes de tabelas e colunas em inglês, `snake_case`, tabelas no plural.
- Cada tabela é um modelo do Django com nome fixo (`db_table`); as restrições ficam em `Meta.constraints` e `Meta.indexes` e viram CHECK e índices de verdade no MySQL.
- Chaves primárias `bigint AUTO_INCREMENT` (`BigAutoField`), exceto `workers.id` (`varchar(200)`: o nome do worker).
- Datas sempre `datetime(6)` em UTC (`USE_TZ = True`); `created_at` e `updated_at` têm padrão no banco (`db_default`) e no Django.
- Chaves estrangeiras com `ON DELETE` no próprio banco (`DB_CASCADE` e `DB_SET_NULL`, do Django 6.1), verificadas na hora pelo InnoDB.
- Estados como `varchar(16)` com `CHECK` (lista fechada), mais simples de evoluir que ENUM.
- Dados livres em `json` (parâmetros, resultados, grafo, detalhes da auditoria); listas também em `json` (arrays), já que o MySQL não tem tipo array.
- Segredos nunca em texto puro: senha com o hasher scrypt do Django, tokens em SHA-256.

Integridade e concorrência

- **Compare-and-set:** toda mudança de estado leva o estado esperado no filtro, como `TaskInstance.objects.filter(id=…, status='queued').update(…)`, ou trava a linha com `select_for_update()` antes de mudar. Quem chega atrasado altera 0 linhas e desiste.
- **Cerca por tentativa:** heartbeat e resultado só valem com `attempts = <número da tentativa>`; um worker que perdeu a posse é ignorado.
- **SKIP LOCKED:** agendamento, promoção de execuções, despacho e recuperação travam linhas com `select_for_update(skip_locked=True)` (`FOR UPDATE SKIP LOCKED`), então vários agendadores dividem o trabalho sem esperar uns pelos outros (MySQL 8).
- **Chave única por horário:** o MySQL não tem índice parcial, então `workflow_runs_schedule_key` é um índice único funcional em `(workflow_id, CASE WHEN trigger = 'schedule' THEN logical_date END)`; nas outras origens a expressão é NULL e não colide. Impede duas execuções agendadas para o mesmo horário; a criação roda num savepoint e o conflito é ignorado.
- **Travas nomeadas:** `GET_LOCK('delco:migrate', -1)` nas migrações e `GET_LOCK('delco:housekeeping', 0)` na limpeza periódica.
- **Publicar depois do COMMIT:** a tarefa vira `queued` no banco antes de a mensagem ir para o Redis; se o processo cair no meio, a recuperação republica.

Tipos usados

Tipo	Onde
`bigint`	Identificadores e chaves estrangeiras

Tipo	Onde
`varchar(n)`	Nomes, estados, filas, e-mail e textos curtos (com tamanho, para poder ter índice)
`longtext`	Descrição, mensagens, erros e tracebacks
`varchar(64)`	SHA-256 em hexadecimal (impressão digital do grafo e hash dos tokens)
`integer` / `smallint`	Contadores, número da tentativa, duração em ms, progresso (0 a 100)
`double`	Segundos de espera, multiplicador e tempo limite
`bool` (`tinyint(1)`)	Ativo, pausado, pedido de cancelamento
`json`	Parâmetros, resultado, grafo da versão, detalhes da auditoria, dependências, filas e etiquetas
`datetime(6)`	Todas as datas, em UTC

Capítulo 4

Tabelas

users

Acesso · modelo `User` · pessoas que entram no painel e na API.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador.
`email`	`varchar(254)`	não		E-mail de login; único sem diferenciar maiúsculas.
`name`	`varchar(120)`	não		Nome exibido.
`password_hash`	`varchar(255)`	não		Formato do Django: `scrypt$sal$N$r$p$hash`.
`role`	`varchar(16)`	não		`admin`, `operator` ou `viewer`.
`is_active`	`bool`	não	true	Conta desativada não entra e perde os tokens.
`failed_logins`	`integer`	não	0	Senhas erradas seguidas.
`locked_until`	`datetime(6)`	sim		Fim do bloqueio por tentativas erradas.
`last_login_at`	`datetime(6)`	sim		Último login com sucesso.
`created_at`, `updated_at`	`datetime(6)`	não	now(6)	Auditoria do registro.

Chaves, índices e restrições

- PK `id`; índice único `users_email_key` em `lower(email)`.
- CHECK em `role` e `failed_logins >= 0`.

Regras

- Após `DELCO_LOGIN_MAX_FAILURES` erros, `locked_until = now() + DELCO_LOGIN_LOCK_MINUTES` e o contador zera.
- Ninguém altera o próprio papel nem desativa a própria conta pela API.

api_tokens

Acesso · modelo `ApiToken` · sessões do painel e tokens pessoais.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador.
`user_id`	`bigint`	não		Dono do token (FK users, CASCADE).
`name`	`varchar(120)`	não		Rótulo: "Sessão no painel" ou o nome dado pelo usuário.

Campo	Tipo	Nulo	Padrão	Descrição
`kind`	`varchar(16)`	não		`session` ou `personal`.
`token_hash`	`varchar(64)`	não		SHA-256 do token; o valor original nunca é gravado.
`prefix`	`varchar(32)`	não		Começo do token (`dlc_xxxxxx`) para reconhecê-lo na lista.
`created_at`	`datetime(6)`	não	now(6)	Criação.
`last_used_at`	`datetime(6)`	sim		Último uso (atualizado no máximo uma vez por minuto).
`expires_at`	`datetime(6)`	sim		Validade; nulo = sem validade (só tokens pessoais).
`revoked_at`	`datetime(6)`	sim		Revogação (logout, troca de senha ou pelo usuário).

Chaves, índices e restrições

- PK `id`; UNIQUE `token_hash`; índice `api_tokens_user_idx`.
- CHECK em `kind`.

Regras

- Token válido = não revogado, não vencido e usuário ativo.
- A limpeza apaga tokens vencidos há 1 dia e revogados há 30 dias.

audit_log

Acesso · modelo `AuditLog` · registro das ações relevantes.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador.
`user_id`	`bigint`	sim		Quem fez (FK users, vira nulo se o usuário for apagado).
`action`	`varchar(64)`	não		`auth.login`, `auth.failed`, `run.trigger`, `run.cancel`, `run.retry`, `workflow.pause`, `token.create`, `user.update`…
`target`	`varchar(255)`	sim		Alvo no formato `tipo:id` (ex.: `run:118`).
`details`	`json`	não	{}	Contexto: parâmetros, tarefas retomadas, campos alterados.
`created_at`	`datetime(6)`	não	now(6)	Quando.

Chaves, índices e restrições

- PK `id`; índice `audit_log_created_idx` (created_at DESC).

Regras

- Só inserções; nunca é alterada.
- Senhas e tokens nunca entram em `details`.
- A limpeza apaga registros com mais de 180 dias.

workflows

Definições · modelo `Workflow` · um registro por workflow encontrado na pasta.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador.
`name`	`varchar(100)`	não		Nome do workflow (único).
`description`	`longtext`	não	''	Descrição.
`schedule`	`varchar(120)`	sim		Cron de 5 campos; nulo = só manual.
`timezone`	`varchar(64)`	não	America/ Sao_Paulo	Fuso da agenda.
`max_active_runs`	`integer`	não	1	Execuções simultâneas permitidas.
`default_params`	`json`	não	{}	Parâmetros padrão.
`tags`	`json`	não	[]	Etiquetas.
`source_file`	`varchar(500)`	sim		Arquivo relativo à pasta de workflows.
`current_version_id`	`bigint`	sim		Versão atual do grafo (FK workflow_versions).
`is_paused`	`bool`	não	false	Agenda pausada pelo operador.
`is_active`	`bool`	não	true	Falso quando o arquivo some da pasta.
`next_run_at`	`datetime(6)`	sim		Próximo horário agendado.
`created_at, updated_at`	`datetime(6)`	não	now(6)	Auditoria do registro.

Chaves, índices e restrições

- PK `id`; UNIQUE `name`; FK `current_version_id` (SET NULL).
- Índice `workflows_due_idx` em `next_run_at` (o filtro de ativo, não pausado e com agenda fica na consulta).

Regras

- Atualizada só pelo agendador (sincronização) e pelas ações de pausa.
- Arquivo com erro de sintaxe não desativa os workflows dele: eles continuam como estavam.
- Mudar agenda ou fuso recalcula `next_run_at`; retomar a pausa recalcula a partir de agora.

workflow_versions

Definições · modelo `WorkflowVersion` · cada grafo diferente que um workflow já teve.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador.
`workflow_id`	`bigint`	não		Workflow (FK, CASCADE).
`version`	`integer`	não		1, 2, 3… por workflow.
`fingerprint`	`varchar(64)`	não		SHA-256 do grafo (tarefas, dependências e opções).
`graph`	`json`	não		Grafo completo: `{workflow, tasks: [{name, upstream, retries, retry_delay, retry_backoff, max_retry_delay, timeout, queue, trigger_rule, description, function, line}]}`.
`created_at`	`datetime(6)`	não	now(6)	Quando a versão apareceu.

Chaves, índices e restrições

- PK `id`; UNIQUE `(workflow_id, version)` e `(workflow_id, fingerprint)`.

Regras

- Imutável. Nova versão só nasce quando a impressão digital muda.
- Cada execução aponta para a versão com que foi criada, então mudar o arquivo não altera execuções em andamento.

workflow_runs

Execução · modelo `WorkflowRun` · cada execução de um workflow.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Número da execução.
`workflow_id`	`bigint`	não		Workflow (FK, CASCADE).
`version_id`	`bigint`	não		Versão do grafo usada (FK).
`status`	`varchar(16)`	não	queued	`queued`, `running`, `succeeded`, `failed`, `cancelled`.
`trigger`	`varchar(16)`	não		`schedule`, `manual` (painel), `api` ou `cli`.
`triggered_by_id`	`bigint`	sim		Usuário que disparou (FK users).
`logical_date`	`datetime(6)`	sim		Horário da agenda que gerou a execução.
`params`	`json`	não	{}	Parâmetros finais (padrões + enviados).
`note`	`longtext`	sim		Observação do operador.
`created_at`	`datetime(6)`	não	now(6)	Criação.
`started_at`	`datetime(6)`	sim		Quando ganhou vaga.

Campo	Tipo	Nulo	Padrão	Descrição
`finished_at`	`datetime(6)`	sim		Quando terminou.
`updated_at`	`datetime(6)`	não	now(6)	Última mudança.

Chaves, índices e restrições

- PK `id`; UNIQUE funcional `workflow_runs_schedule_key (workflow_id, CASE WHEN trigger = 'schedule' THEN logical_date END)`.
- Índices: `(workflow_id, id DESC)`, `(status, id)` para abertas e `finished_at` para a limpeza.

Regras

- Nasce `queued` com todas as tarefas `pending`; vira `running` quando há vaga em `max_active_runs`.
- Termina quando todas as tarefas terminam: `failed` se alguma falhou, `cancelled` se alguma foi cancelada, senão `succeeded`.
- Retomar volta a execução para `queued` e as tarefas escolhidas para `pending`.
- Apagada (com tudo abaixo) após `DELCO_RETENTION_DAYS` do fim.

task_instances

Execução · modelo `TaskInstance` · cada tarefa de cada execução.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador (vai na mensagem da fila).
`run_id`	`bigint`	não		Execução (FK, CASCADE).
`task_name`	`varchar(100)`	não		Nome da tarefa no grafo.
`status`	`varchar(16)`	não	pending	Veja a máquina de estados.
`upstream`	`json`	não	[]	Dependências (copiadas do grafo).
`trigger_rule`	`varchar(16)`	não	all_success	`all_success`, `all_done` ou `one_failed`.
`queue`	`varchar(100)`	não	default	Fila onde é publicada.
`retries`	`integer`	não	2	Retentativas configuradas.
`max_attempts`	`integer`	não	3	Limite atual de tentativas (cresce a cada retomada manual).
`retry_delay`, `retry_backoff`, `max_retry_delay`	`double`	não	5, 2, 600	Política de espera.
`timeout_seconds`	`double`	sim		Tempo limite por tentativa.
`attempts`	`integer`	não	0	Tentativas iniciadas até agora.
`worker_id`	`varchar(200)`	sim		Worker com a posse atual (FK workers).
`lease_expires_at`	`datetime(6)`	sim		Validade da posse, renovada a cada heartbeat.

Campo	Tipo	Nulo	Padrão	Descrição
`queue_message_id`	`varchar(64)`	sim		Id da mensagem no Redis Stream.
`cancel_requested`	`bool`	não	false	Pedido de parada lido no heartbeat.
`progress`	`smallint`	sim		0 a 100, informado por `ctx.progress`.
`output`	`json`	sim		Resultado da tarefa (até 1 MB).
`error`	`longtext`	sim		Último erro.
`ready_at`	`datetime(6)`	sim		Quando a nova tentativa pode ir para a fila.
`queued_at`, `started_at`, `finished_at`	`datetime(6)`	sim		Marcos da tarefa.
`updated_at`	`datetime(6)`	não	now(6)	Última mudança.

Chaves, índices e restrições

- PK `id`; UNIQUE `(run_id, task_name)`.
- Índices: `(status, run_id)` para estados abertos e `(status, lease_expires_at)` para os leases de `running`.
- CHECK em `status`, `trigger_rule`, `progress` e contadores.

Regras

- Assumir: `select_for_update()` na tarefa com `status='queued'` e execução `running`; só depois soma a tentativa e grava `running`.
- Heartbeat e fim só valem com `status='running' AND attempts=<tentativa>`.
- Lease vencido: a tentativa vira `lost` e a tarefa vai para `retry_wait` (ou `failed` sem tentativas restantes).

task_attempts

Execução · modelo `TaskAttempt` · cada tentativa de uma tarefa.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador (usado pelos logs e pelo WebSocket).
`task_instance_id`	`bigint`	não		Tarefa (FK, CASCADE).
`number`	`integer`	não		1, 2, 3… (igual a `attempts` no momento em que começou).
`worker_id`	`varchar(200)`	sim		Worker que executou.
`status`	`varchar(16)`	não	running	`running`, `succeeded`, `failed`, `timed_out`, `lost`, `cancelled`.
`started_at`	`datetime(6)`	não	now(6)	Início.
`finished_at`	`datetime(6)`	sim		Fim.

Campo	Tipo	Nulo	Padrão	Descrição
`duration_ms`	`integer`	sim		Duração.
`error`	`longtext`	sim		Tipo e mensagem do erro.
`traceback`	`longtext`	sim		Traceback completo do Python.

Chaves, índices e restrições

- PK `id`; UNIQUE `(task_instance_id, number)`.

Regras

- Criada na mesma transação em que a tarefa é assumida.
- `lost` = o worker sumiu ou foi desligado antes de terminar.

task_logs

Execução · modelo `TaskLog` · linhas de log de cada tentativa.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Ordem das linhas.
`attempt_id`	`bigint`	não		Tentativa (FK, CASCADE).
`logged_at`	`datetime(6)`	não	now(6)	Quando a linha foi escrita.
`level`	`varchar(16)`	não		`debug`, `info`, `warning`, `error`.
`message`	`longtext`	não		Texto (até 8000 caracteres por linha).

Chaves, índices e restrições

- PK `id`; índice `(attempt_id, id)` para leitura paginada.

Regras

- Gravada em lotes de até 50 linhas ou a cada 0,4 s.
- Limite de `DELCO_MAX_LOG_LINES` linhas por tentativa, com um aviso de truncamento.

workers

Infraestrutura · modelo `Worker` · processos worker conhecidos.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`varchar(200)`	não		Nome: `DELCO_WORKER_NAME` ou host + pid.
`hostname`	`varchar(255)`	não		Máquina ou contêiner.
`pid`	`integer`	não		Processo.
`queues`	`json`	não		Filas consumidas.

Campo	Tipo	Nulo	Padrão	Descrição
`concurrency`	`integer`	não		Vagas.
`version`	`varchar(32)`	não		Versão do Delco.
`status`	`varchar(16)`	não	online	`online`, `stopping`, `offline`.
`started_at`	`datetime(6)`	não	now(6)	Subida.
`last_heartbeat_at`	`datetime(6)`	não	now(6)	Último sinal.
`stopped_at`	`datetime(6)`	sim		Quando parou.

Chaves, índices e restrições

- PK `id`.

Regras

- Registro com `bulk_create(update_conflicts=True)` (`INSERT ... ON DUPLICATE KEY UPDATE`) a cada subida.
- Sem sinal por 30 s vira `offline`; apagado após 7 dias offline.

django_migrations

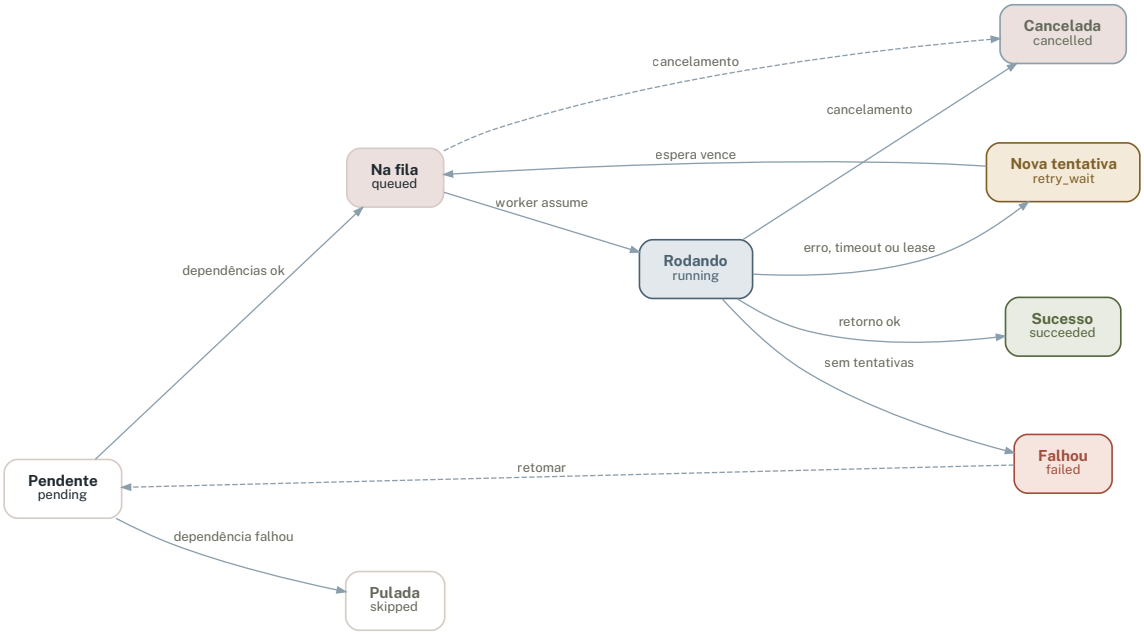
Infraestrutura · controle das migrações aplicadas, criada e mantida pelo próprio Django.

Campo	Tipo	Nulo	Padrão	Descrição
`id`	`bigint`	não	auto_increment	Identificador.
`app`	`varchar(255)`	não		Aplicação Django (`delco`).
`name`	`varchar(255)`	não		Migração (ex.: `0001_inicial`).
`applied`	`datetime(6)`	não		Quando foi aplicada.

Capítulo 5

Máquinas de estado

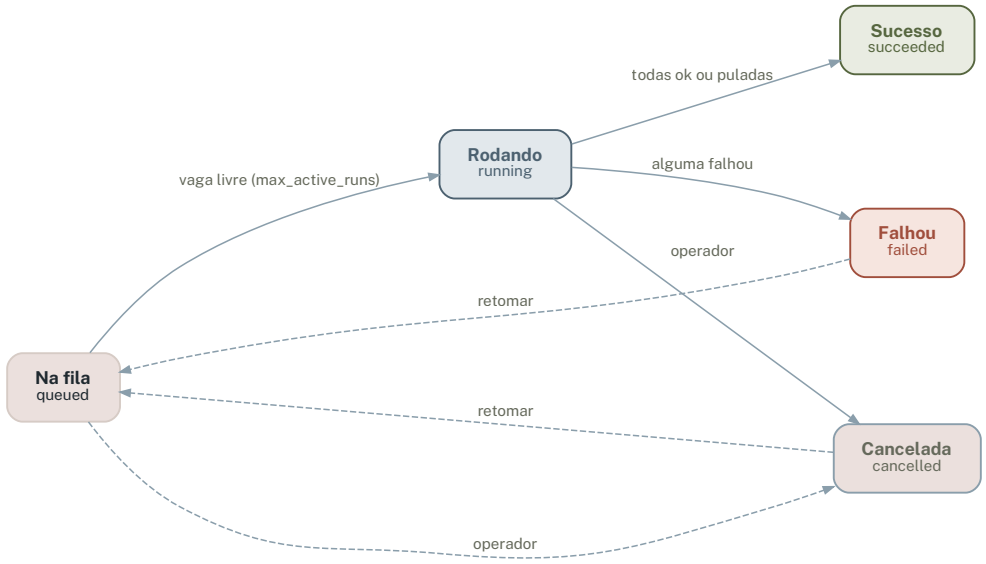
Tarefa



task_instances.status

De	Para	Quem	Condição no SQL
pending	queued	agendador	dependências satisfazem a regra; `status IN ('pending','retry_wait')`
pending	skipped	agendador	dependência terminou sem sucesso (regra all_success) ou nada falhou (one_failed)
queued	running	worker	`status='queued'` e execução `running`
running	succeeded / retry_wait / failed	worker	`status='running' AND attempts=n`
running	retry_wait / failed	agendador	`lease_expires_at < now(6)`
retry_wait	queued	agendador	`ready_at <= now(6)`
pending, queued, retry_wait	cancelled	API	cancelamento da execução
running	cancelled	worker	`cancel_requested` lido no heartbeat
failed, skipped, cancelled	pending	API	retomar a execução

Execução



workflow_runs.status

- **queued → running:** o agendador conta as execuções `running` do workflow e promove as mais antigas até `max_active_runs`.
- **running → succeeded / failed / cancelled:** quando nenhuma tarefa está em `pending`, `queued`, `running` ou `retry_wait`.
- **queued/running → cancelled:** pelo operador; tarefas rodando recebem `cancel_requested`.
- **failed/cancelled → queued:** retomar; também vale para `succeeded` ao reexecutar a partir de uma tarefa.

Capítulo 6

Uma execução passo a passo

1. **Disparo:** `INSERT INTO workflow_runs` (status `queued`) e um `INSERT INTO task_instances` por tarefa do grafo, tudo na mesma transação, mais um registro em `audit_log`.
2. **Vaga:** o agendador trava o workflow (`FOR UPDATE SKIP LOCKED`), conta as execuções rodando e muda a execução para `running`.
3. **Despacho:** trava a execução, lê os estados das tarefas, marca as prontas como `queued` e as impossíveis como `skipped`. Depois do COMMIT, publica cada id no Redis e grava `queue_message_id`.
4. **Posse:** o worker faz o compare-and-set para `running`, incrementa `attempts`, define o lease e insere `task_attempts`; lê parâmetros e resultados das dependências.
5. **Execução:** o processo filho roda a função; logs vão para `task_logs` em lotes; o heartbeat renova o lease e grava o progresso.
6. **Fim:** o worker trava a tarefa (com a cerca de tentativa), fecha a tentativa e grava `succeeded` com o resultado, `retry_wait` com `ready_at` ou `failed`.
7. **Fechamento:** o agendador é acordado, libera as próximas tarefas e, quando todas terminam, grava o estado final da execução e promove a próxima da fila.

Consultas úteis

```
-- execuções com falha nas últimas 24 horas
SELECT r.id, w.name, r.finished_at
  FROM workflow_runs r JOIN workflows w ON w.id = r.workflow_id
 WHERE r.status = 'failed' AND r.finished_at > NOW() - INTERVAL 24 HOUR
 ORDER BY r.id DESC;

-- tarefas que mais falham (tentativas) nos últimos 7 dias
SELECT w.name, t.task_name, count(*) AS falhas
  FROM task_attempts a
  JOIN task_instances t ON t.id = a.task_instance_id
  JOIN workflow_runs r ON r.id = t.run_id
  JOIN workflows w ON w.id = r.workflow_id
 WHERE a.status IN ('failed', 'timed_out', 'lost') AND a.finished_at > NOW() - INTERVAL 7 DAY
 GROUP BY 1, 2 ORDER BY falhas DESC LIMIT 10;

-- o que está rodando agora e em qual worker
SELECT t.run_id, t.task_name, t.worker_id, t.progress, t.lease_expires_at
  FROM task_instances t WHERE t.status = 'running' ORDER BY t.started_at;

-- duração média por workflow (execuções com sucesso, 30 dias)
SELECT w.name, ROUND(AVG(TIMESTAMPDIFF(MICROSECOND, r.started_at, r.finished_at)) / 1e6, 1) AS
segundos
  FROM workflow_runs r JOIN workflows w ON w.id = r.workflow_id
 WHERE r.status = 'succeeded' AND r.created_at > NOW() - INTERVAL 30 DAY
 GROUP BY 1 ORDER BY 2 DESC;
```

Capítulo 7

Como parear o banco

Com Docker (recomendado)

O `docker-compose.yml` já sobe o MySQL 8.4 com volume persistente `mysql-data`, usuário, senha e banco do `.env`. A API aplica as migrações ao subir. Para abrir um terminal SQL:

```
docker compose exec mysql sh -c 'mysql -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE"'
docker compose exec api python manage.py dbshell    # o mesmo, pelo Django
```

A porta é publicada só em `127.0.0.1:${DELCO_DB_PORT}`, para clientes como DBeaver ou MySQL Workbench na sua máquina.

MySQL próprio

1. Crie o usuário e o banco:

   ```
   CREATE DATABASE delco CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
   CREATE USER 'delco'@'%' IDENTIFIED BY 'troque-esta-senha';
   GRANT ALL PRIVILEGES ON delco.* TO 'delco'@'%';
   ```

2. Aponte o Delco para ele: `DELCO_DATABASE_URL=mysql://delco:SENHA@servidor:3306/delco` (no Docker, troque a variável no serviço ou remova o serviço `mysql` do Compose).
3. Crie as tabelas: `delco migrate` (ou `python manage.py migrate`). O comando é idempotente: rodar de novo não altera nada.
4. Se usar SSL, acrescente `?ssl_mode=REQUIRED` à URL.

Banco gerenciado

Qualquer MySQL 8.4 ou mais novo na nuvem funciona (Amazon RDS, Aurora MySQL, Google Cloud SQL, Azure Database for MySQL). O usuário precisa poder criar tabelas no banco (só na primeira migração) e usar `GET_LOCK`, que é padrão. Cada processo abre uma conexão por vaga; se o provedor limitar conexões, some os workers × `DELCO_WORKER_CONCURRENCY` mais a API e o agendador.

Redis

Redis 6.2 ou mais novo (por causa do `XAUTOCLAIM`). No Compose ele roda com AOF ligado para não perder a fila num reinício, mas mesmo sem persistência nada se perde. URL em `DELCO_REDIS_URL`.

Backup e restauração

```
# backup
docker compose exec -T mysql sh -c 'mysqldump -u"$MYSQL_USER" -p"$MYSQL_PASSWORD"
--single-transaction --no-tablespaces "$MYSQL_DATABASE"' > delco-$(date +%F).sql

# restauração num banco vazio
docker compose exec -T mysql sh -c 'mysql -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE"' <
delco-2026-10-20.sql
```

Faça backup só do MySQL. Depois de restaurar, suba os processos normalmente: tarefas que estavam `running` no momento do backup serão recuperadas pelo vencimento do lease.

Retenção e tamanho

- Execuções terminadas há mais de `DELCO_RETENTION_DAYS` (30) são apagadas com tarefas, tentativas e logs.
- `task_logs` costuma ser a maior tabela. Reduza `DELCO_MAX_LOG_LINES` ou a retenção se o banco crescer demais.
- O InnoDB reaproveita o espaço sozinho; em volumes altos, rode `OPTIMIZE TABLE task_logs` depois de grandes limpezas.

Problemas comuns

Mensagem	Solução
Access denied for user 'delco'	Senha do `.env` diferente da usada na criação do volume. Ajuste ou apague o volume (`docker compose down -v` , perde os dados).
Table 'delco.workflows' doesn't exist	Migração não rodou: `delco migrate` ou `python manage.py migrate` .
Can't connect to MySQL server	Banco fora do ar ou ainda subindo. Confira `docker compose ps` e os logs do serviço `mysql` .
Too many connections	Muitos processos para o limite do servidor. Aumente `max_connections` ou reduza workers.
Execuções agendadas não aparecem	Agendador parado ou workflow pausado. Confira `next_run_at` e `is_paused` em `workflows` .

Capítulo 8

Evolução do esquema

O esquema nasce dos modelos em `delco/models.py` e as migrações ficam em `delco/migrations/`, geradas pelo Django e registradas em `django_migrations`. Para mudar o esquema:

1. Altere o modelo em `delco/models.py` (campo, índice ou restrição em `Meta`).
2. Gere a migração: `python manage.py makemigrations delco --name indice_logs`.
3. Confira o SQL que será executado: `python manage.py sqlmigrate delco 0002`.
4. Prefira mudanças compatíveis: colunas novas com padrão (`db_default`) ou nulas; em tabelas grandes, o InnoDB cria índices sem bloquear escritas (DDL online).
5. Aplique com `delco migrate` (ou suba a API): o atalho usa a trava nomeada `delco:migrate`, então vários processos podem subir juntos.
6. Atualize os testes e este documento. O CI falha se um modelo mudar sem migração (`makemigrations --check`).

```
# delco/models.py, dentro de class TaskLog.Meta
indexes = [
    models.Index(fields=["attempt", "id"], name="task_logs_attempt_idx"),
    models.Index(fields=["logged_at"], name="task_logs_logged_at_idx"),   # novo
]
```